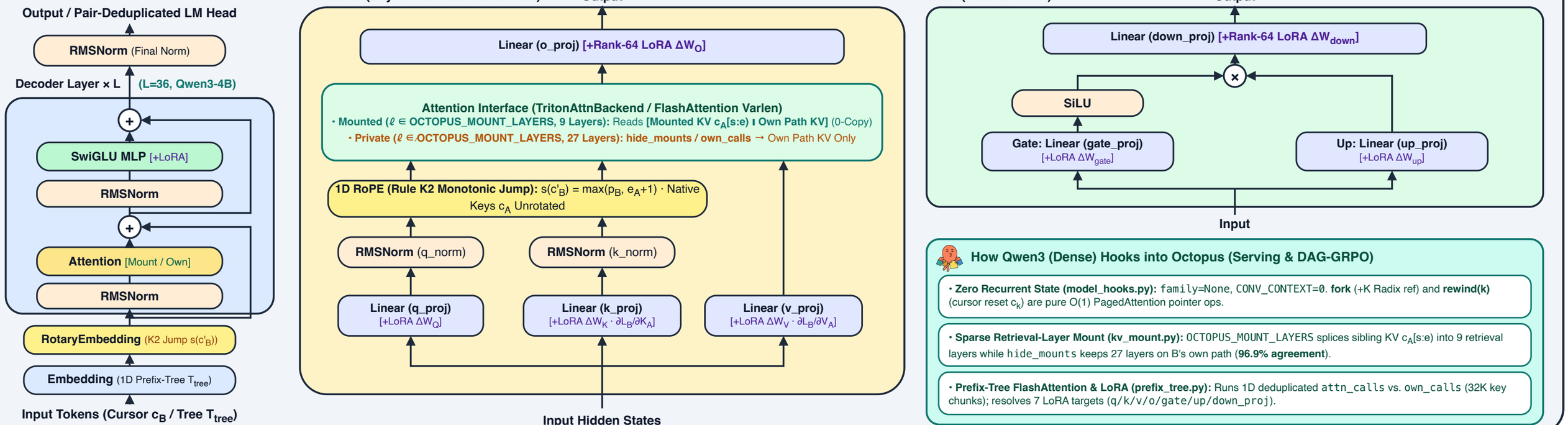
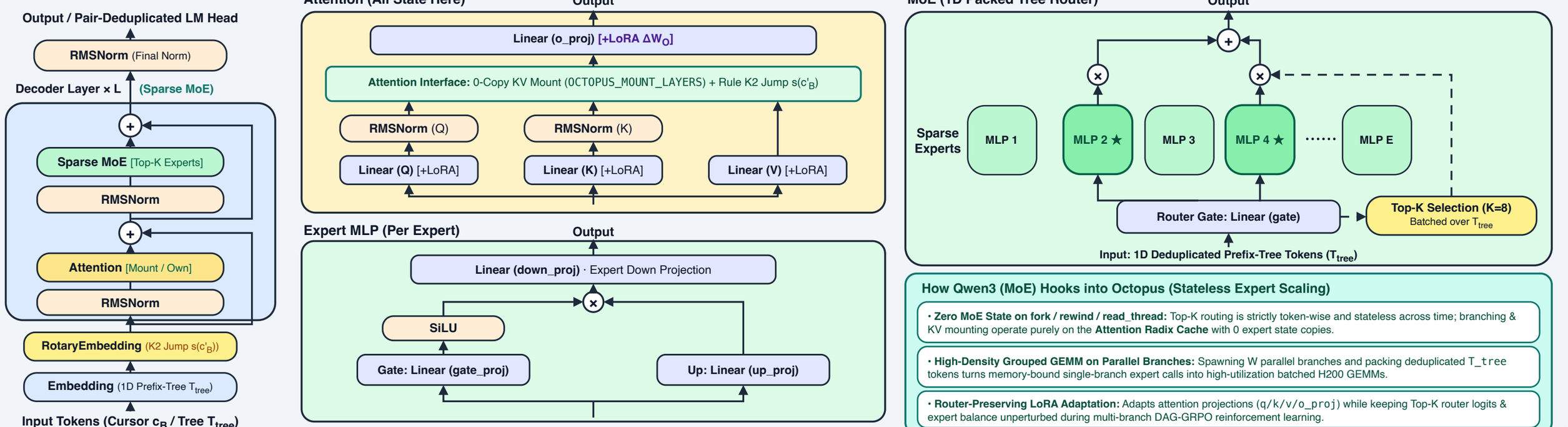
(a) Qwen3 (Dense) Architecture & Octopus Integration Blueprint (Pure Full-Attention · Sparse 9-Layer KV Mount · 96.9% Agreement)



How Qwen3 (Dense) Hooks into Octopus (Serving & DAG-GRPO)

- **Zero Recurrent State (model_hooks.py):** `family=None`, `CONV_CONTEXT=0`. **fork** (+K Radix ref) and **rewind(k)** (cursor reset c_k) are pure O(1) PagedAttention pointer ops.
- **Sparse Retrieval-Layer Mount (kv_mount.py):** `OCTOPUS_MOUNT_LAYERS` splices sibling KV c_A[s:e) into 9 retrieval layers while `hide_mounts` keeps 27 layers on B's own path (**96.9% agreement**).
- **Prefix-Tree FlashAttention & LoRA (prefix_tree.py):** Runs 1D deduplicated `attn_calls` vs. `own_calls` (32K key chunks); resolves 7 LoRA targets (`q/k/v/o/gate/up/down_proj`).

(b) Qwen3 (MoE) Architecture & Octopus Integration Blueprint (Sparse Top-K MoE · Stateless 1D Prefix-Tree Grouped GEMM · 0 Expert State)



How Qwen3 (MoE) Hooks into Octopus (Stateless Expert Scaling)

- **Zero MoE State on fork / rewind / read_thread:** Top-K routing is strictly token-wise and stateless across time; branching & KV mounting operate purely on the **Attention Radix Cache** with 0 expert state copies.
- **High-Density Grouped GEMM on Parallel Branches:** Spawning W parallel branches and packing deduplicated `T_tree` tokens turns memory-bound single-branch expert calls into high-utilization batched H200 GEMMs.
- **Router-Preserving LoRA Adaptation:** Adapts attention projections (`q/k/v/o_proj`) while keeping Top-K router logits & expert balance unperturbed during multi-branch DAG-GRPO reinforcement learning.